

Railroad North

Applied OT & ICS Security Training

A Hands-On Workshop for Engineering & Cyber Security Students

Workshop Overview

This is an **intensive hands-on workshop** that puts you inside a live OT environment.

Part 1: Guided Exploration

Understand OT/ICS fundamentals. Explore the SCADA dashboard. Execute supervised attack and defense scenarios against a live railway control system.

Part 2: Capture The Flag

Apply everything you learned. Work in teams to solve 10 challenges across Reconnaissance, Exploitation, Forensics, and Defense categories using Wireshark and the lab tools.

What is Operational Technology (OT)?

Unlike traditional IT networks that manage **data**, OT networks manage **physical processes**.

Traditional IT (Information Tech)

- **Priority:** Confidentiality & Data Protection
- **Impact:** Data breaches, financial loss
- **Lifespan:** Hardware replaced every 3-5 years
- **Patching:** Regular, automated updates

OT (Operational Tech)

- **Priority:** **Safety, Availability, Reliability**
- **Impact:** Physical damage, loss of human life
- **Lifespan:** Equipment runs for 15-30 years
- **Patching:** Extremely rare; requires system downtime

Introduction to the Railroad North Lab

This laboratory simulates a **Critical Infrastructure Railway System**. It is designed to safely teach you how these systems work and how they can be compromised.

The Core Mission You will step into the shoes of both an attacker and a defender. You will learn how to read industrial protocols, bypass physical safety interlocks, and detect these attacks using network forensics and log correlation.

Lab Architecture & Components

Our environment runs **8 Docker containers** across 3 segmented networks:

Container	Role	Port	Network
SCADA Dashboard	Operator Interface	localhost:8081	DMZ + OT
Master PLC	Central Coordinator	localhost:8085	OT
Slave PLC 1 (North)	Field Controller	localhost:8086	OT
Slave PLC 2 (Central)	Field Controller	localhost:8087	OT
Slave PLC 3 (South)	Field Controller	localhost:8088	OT
Syslog Collector	Log Aggregation	localhost:5140	OT
Wireshark	Packet Analysis	localhost:3000	OT
CTF Server	Challenge Scoring	localhost:8090	DMZ + OT

The Purdue Enterprise Reference Architecture

Our lab is segmented strictly according to the industry-standard Purdue Model.

Zone	Level	Components in our Lab
IT / Enterprise	Level 4	Your laptop (the attacker's workstation)
DMZ (Demilitarized)	Level 3.5	SCADA Web Dashboard, CTF Scoreboard (172.27.x.x)
OT / Control	Level 1-3	Master PLC, Syslog Collector, Wireshark (172.25.x.x)
Field Devices	Level 0	Slave PLCs controlling Switches, Barriers, and Signals

Deep Dive: How the Railway is Controlled

To secure the railway, you must first understand how it operates.

1. The Master PLC (The Brain)

The central coordinator. It stores the "global truth" of the railway. It continuously polls the Slave PLCs every 5 seconds via a **Heartbeat** and enforces safety interlock rules before allowing any track switch.

2. The Slave PLCs (The Muscle)

Located in the North, Central, and South segments. These directly control physical track switches, signals, and barriers. They report sensor data to the Master and execute its commands.

API Communication: The Attack Surface

Our PLCs communicate using **REST APIs over HTTP** (simulating the insecurity of real-world Modbus TCP).

The Security Flaw: The API endpoints have zero authentication. If you can reach the network port, the PLC will execute your command without verifying who sent it.

Key API Endpoints You Will Target:

- `GET /api/status` - Read the full system state (reconnaissance)
- `POST /api/command` - Switch a track route (exploitation)
- `POST /api/device/set` - Manipulate field sensors (sensor spoofing)
- `POST /api/emergency` - Trigger an emergency stop

Built-In Safety Interlocks (Defensive Logic)

To prevent catastrophic train derailments, the Master PLC has hard-coded safety logic:

1. **Route Conflict Prevention:** The Master PLC checks if two connecting segments are requesting conflicting routes at a junction. If they conflict, the command is **rejected**.
2. **Track Occupancy Lock:** A track switch will not operate if the occupancy sensor detects a train currently on that segment.
3. **Barrier Enforcement:** Routes cannot be cleared unless physical crossing barriers are confirmed to be lowered and locked.
4. **Heartbeat Monitoring:** If the Master PLC loses connection to a Slave PLC (3 missed heartbeats), it triggers a global **EMERGENCY STOP**.

PART 1

Guided Attack & Defense Scenarios

Scenario 1: Unauthorized Track Switching

Concept: An adversary bypasses the SCADA dashboard and sends a direct API command to switch a track route.

The Attack

Run `python scripts/modbus-attack.py` and select **Option 1**. This sends a direct POST request to the Master PLC, bypassing the SCADA operator entirely.

The Defense

Watch the SCADA Dashboard. The System Audit Log will show the command in **yellow** (UNAUTHORIZED API CALL) instead of green (legitimate operator command). This is **Log Correlation** in action.

Scenario 2: Heartbeat Failure (DoS Attack)

Concept: A segment of the railway loses connectivity due to a Denial of Service attack or a physical network cut.

The Attack

Run `docker stop railroad-slave-plc-1` in your terminal. This simulates cutting the network cable to the North segment. Wait 15-20 seconds.

The Defense

Watch the dashboard enter **EMERGENCY STOP** automatically. The Master PLC detected 3 missed heartbeats and triggered the fail-safe. The system "fails closed" for physical safety.

Recovery: `docker start railroad-slave-plc-1`, wait 5 seconds, then click **Clear Faults** on the SCADA dashboard.

Scenario 3: SCADA Alarm Flooding

Concept: The attacker overwhelms the SCADA operator by flooding the system with rapid, conflicting commands.

The Attack

Run `python scripts/modbus-attack.py` and select **Option 3**. This fires 50 commands in 5 seconds, rapidly toggling a track switch back and forth.

The Defense

The Audit Log is overwhelmed with yellow warnings. In a real SOC, this is called **Alert Fatigue**. Defenders must implement rate limiting and anomaly detection to filter the noise from genuine threats.

Scenario 4: Safety Interlock Validation

Concept: Verify that the safety interlocks actually prevent dangerous operations even under attack.

The Test

Set Segment 1 to ROUTE_A, then attempt to set Segment 2 to ROUTE_A. The Master PLC should **reject** this because the routes conflict at the junction. The Audit Log will show a **red REJECTED** entry with the reason.

The Takeaway

Even if an attacker gains full API access, the **hard-coded safety logic** in the PLC firmware provides the last line of defense. This is why OT security uses a **defense-in-depth** strategy.

Using Wireshark for Network Forensics

Open **Wireshark** at `http://localhost:3000` in your browser.

What to Look For

- **HTTP POST requests** to `/api/command` -- These are route switch commands
- **Rapid request bursts** -- Indicates a flood/DoS attack
- **Unauthorized source IPs** -- Commands not from the SCADA server (172.27.0.20)
- **Heartbeat gaps** -- Missing periodic GET requests from Slave PLCs

Practice capturing traffic while a partner runs the attack scripts. You will need these skills for the CTF.

PART 2

Capture The Flag Challenge

CTF Rules & Setup

How it Works

- Open the **CTF Scoreboard** at <http://localhost:8090>
- Register your team name
- Solve challenges and submit flags to earn points
- 10 challenges across 4 categories
- Total possible score: **2,400 points**

Categories

- **Reconnaissance (200 pts)**: Map the OT network
- **Exploitation (850 pts)**: Execute attacks and fuzz APIs
- **Forensics (1,100 pts)**: Analyze PCAPs and memory
- **Defense (550 pts)**: Recover and validate

CTF Tools Available

You have access to these tools during the CTF:

Tool	Access	Purpose
SCADA Dashboard	<code>localhost:8081</code>	Monitor the system state in real time
Wireshark	<code>localhost:3000</code>	Capture and analyze network traffic
Attack Script	<code>python scripts/modbus-attack.py</code>	Pre-built attack vectors
curl / browser	Your terminal	Craft custom API requests
Docker CLI	Your terminal	Start/stop containers
Master PLC API	<code>localhost:8085/api/status</code>	Directly query system state
Slave PLC APIs	<code>localhost:8086-8088</code>	Directly query field devices

CTF Challenge Preview

#	Challenge	Category	Points	Difficulty
1	Network Reconnaissance	Recon	100	Easy
2	Protocol Identification	Recon	100	Easy
3	Unauthorized Track Switch	Exploit	200	Medium
4	Sensor Spoofing	Exploit	250	Medium
5	Log Correlation	Forensics	300	Hard
6	Packet Capture Analysis	Forensics	300	Hard
7	Emergency Recovery	Defense	200	Medium
8	Safety Interlock Validation	Defense	350	Hard
9	API Fuzzing	Exploit	400	Insane
10	Deep Memory Forensics	Forensics	500	Insane

Hints are available but cost -50 points each!

Lab Access Summary

What	Where
SCADA Dashboard	<code>http://localhost:8081</code>
CTF Scoreboard	<code>http://localhost:8090</code>
Wireshark	<code>http://localhost:3000</code>
Master PLC API	<code>http://localhost:8085</code>
Slave PLC 1 (North)	<code>http://localhost:8086</code>
Slave PLC 2 (Central)	<code>http://localhost:8087</code>
Slave PLC 3 (South)	<code>http://localhost:8088</code>
Syslog	<code>localhost:5140 (UDP)</code>
Attack Script	<code>python scripts/modbus-attack.py</code>

Questions & Lab Kickoff

Please navigate to your local lab instance to begin.

The TA will be walking around to assist with networking or Docker issues.